

minimize the negative log-likelihood (NLL) of each token $x_{i,j}$ given previous tokens $x_{i,<j}$ in a text sequence $x_i = (x_{i,1}, \dots, x_{i,|j|})$ drawn from D_{train} :

$$\ell_i(\theta) = -\frac{1}{|x_i|} \sum_{j=1}^{|x_i|} \log p_{\theta}(x_{i,j} | x_{i,<j}). \quad (1)$$

Since $\ell_i(\theta)$ directly measures how the model memorizes each token in x_i , we will track it in the pretraining process and identify the sample with consistently small and converged loss across checkpoints as memorized. Specifically, for a sample x_i with loss $\ell_i(\theta_t)$ at pretraining step t , we identify it as *memorized* since step t_i^* if $\ell_i(\theta_t) \leq \varepsilon$ and $|\ell_i(\theta_t) - \ell_i(\theta_T)| \leq \delta$ for all $t \geq t_i^*$, where ε is a small threshold, δ enforces stability over time, and T indexes the final pretraining checkpoint. In Appendix E.3, We verify that varying ε and δ does not change the post-memorization behavior analyzed in Section 4.

Generalization is measured on standard benchmarks after instruction tuning. As foundation models with various emergent capabilities, LLMs are expected to generalize to an open set of unseen tasks through simple prompting/instructions. Hence, their generalization performance is usually measured across standard benchmarks defined on diverse downstream tasks. To equip a pretrained LLM θ with the instruction-following capability, an instruction tuning algorithm $\mathcal{A}_{\text{IT}}$ further finetunes θ on a dataset D_{IT} . The generalization performance on benchmark- i is then measured by

$$\mathcal{G}_i = \text{Eval}_i(\theta_{\text{IT}}, D_i), \theta_{\text{IT}} = \mathcal{A}_{\text{IT}}(\theta, D_{\text{IT}}), \quad (2)$$

where D_i is the dataset in benchmark- i and Eval_i defines the corresponding evaluation metric, which varies across benchmarks for different types of tasks. For example, accuracy is usually used in multi-choice QA tasks while BLEU/ROUGE scores are used in summarization and generation tasks. For code generation tasks, the generalization performance is usually measured by unit tests, where a test case is considered correct if the generated code can pass the unit tests.

Grokking refers to delayed generalization improvement after memorization is completed. Following the same spirit of previous works on grokking, we mainly focus on the phenomenon that $\mathcal{G}_i$ starts to surge long after $\ell_i(\theta)$ has converged/plateaued. We also use the training loss to track memorization as previous works. However, the test loss adopted by previous grokking analysis is no longer an ideal metric for LLM generalization, since it focuses on token-level matching to ground truths while downstream tasks focus on semantic-level matching, and some of them do not provide ground truths (e.g., code generation).

Experimental Setup We equidistantly sample 10 checkpoints from the OLMoE pretraining trajectory. To evaluate memorization, we probe OLMoE on samples drawn directly from its pretraining corpus D_{train} , ensuring fidelity to the model’s original distribution. We evaluate generalization on widely used benchmarks spanning four domains: *math*, *code*, *commonsense QA*, and *domain-specific QA* (10k pretraining and 10k test samples per domain; see Appendix A). To exclude contaminated benchmark samples from the generalization analysis, we apply the Min-K%++ membership inference method [35] to identify them. Moreover, we mainly focus on the samples whose model predictions become consistently correct before the pretraining ends, and analyze when this generalization emerges and how long it persists. For downstream tasks, we apply lightweight LoRA-based instruction tuning $\mathcal{A}_{\text{IT}}$ on D_{IT} to equip checkpoints with basic instruction-following capability while preserving pretrained capabilities (Appendix B).

3.2 Asynchronous Memorization, Delayed Generalization, & Local Grokking

To understand when memorization unfolds during pretraining and how it affects generalization, Figure 1 contrasts the number of memorized training samples (using the criterion defined above) at each pretraining checkpoint with benchmark performance for each domain. It shows that (1) training samples are memorized at different pretraining steps; (2) generalization typically follows memorization with a lag: benchmark performance starts to achieve stable gains after sufficient data has been memorized. An interesting discovery is that the lag length varies across domains: generalization leap on math and coding tasks requires memorizing many more samples than commonsense and domain-specific QA tasks.

These observations reveal the complex training dynamics of LLMs caused by training data heterogeneity and their uneven attributions to different benchmark tasks. It implies that the widely studied

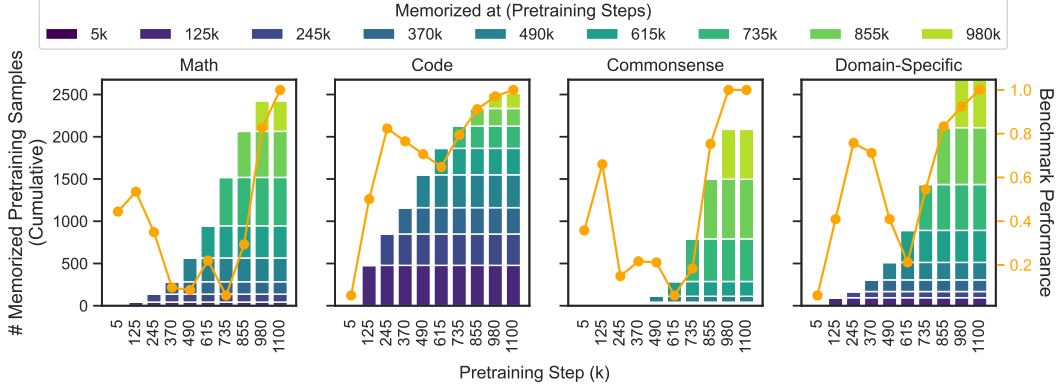


Figure 1: **Memorized training samples at different pretraining steps and the generalization performance** on standard benchmarks for each domain. It shows asynchronous memorization of different data in each domain and a delayed generalization leap after memorizing a certain amount of data.

global grokking—a synchronous memorization of most training data and a delayed contemporary generalization leap on most test samples—cannot be found in LLM pretraining. That being said, the memorization of certain data is plausible to impose a long-term, delayed effect on the generalization to specific downstream tasks. To investigate such local dynamic patterns, we group data in each domain by their memorization steps t_i^* and particularly focus on the early-memorized groups, as they leave more checkpoints after t_i^* to study the subsequent dynamics of generalization. Accordingly, we group samples in benchmarks by the training steps since when their predictions become consistently correct. To enable an analysis of how memorization drives downstream generalization performance, we pair each training data group with its most relevant benchmark samples via bipartite graph matching (by Hungarian algorithm [13]), with the training-test data similarity defined in the embedding space of SentenceTransformer [27]. Additional implementation details of the pairing procedure are provided in Appendix B.1.

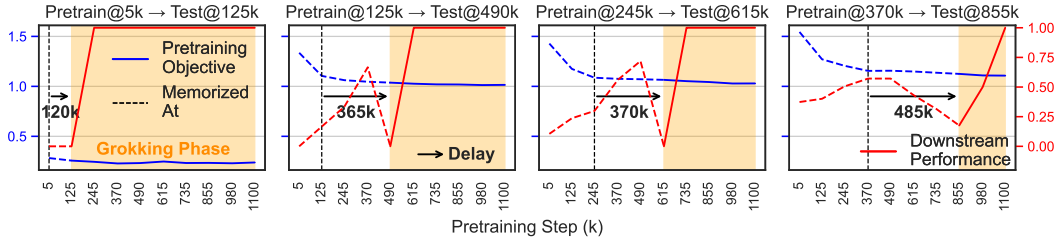


Figure 2: **Pretraining objective (blue) and benchmark performance (red) across four paired training-test data groups from the math domain.** Pretrain@ $p \rightarrow$ Test@ q denotes a paired training–test group, where Pretrain@ p is the training subset whose samples reach stable memorization at step p , and Test@ q is the benchmark subset whose accuracy stabilizes at step q . These pairs are formed using embedding-based Hungarian matching, as detailed in Appendix B.1. We observe local grokking on each pair: a delayed generalization leap on benchmark tasks after the pretraining objective plateaued. From left to right, more difficult data memorized (converged) later associate with a longer delay towards grokking.

Figure 2 visualizes the memorization and generalization on four groups: all training-test pairs exhibit a delayed generalization leap on benchmark tasks after the pretraining objective stably converged on the training data. Hence, the training objective cannot monitor or predict the generalization performance. Moreover, earlier-converging groups generalize sooner after loss converged, while later-converged groups associate with substantially longer delays. Notably, this delay scales with data difficulty, which affects not only the number of steps taken to memorize the data but also the duration of memorization to generalization transition.

Remaining Question This local grokking indicates that the model’s internal states, except its pretraining loss, might track generalization or reflect the transition better. This raises a fundamental question for understanding the emergence of capability in LLM: *What internal state changes within an LLM signify the emergence of generalization?* Investigating this problem can provide critical insights to uncover the mechanisms of memorization-to-generalization transition and monitor the pretraining progress in practice. To this end, we probe LLMs’ internal state dynamics to address the challenge.

4 Monitor Memorization-to-Generalization Transition by Training Dynamics of Routing Pathways

As shown in Section 3, generalization in LLMs emerges well after memorization, suggesting that internal reorganization underlies this transition. Such dynamics are difficult to track in dense architectures, where all neurons are simultaneously involved. MoEs simplify this process: by organizing computation into experts, each input activates only a subset of experts, forming a discrete *routing pathway* across layers (Figure 3(a)). These pathways reveal how computation is allocated and reorganized during pretraining, providing a mechanistic view of the memorization-to-generalization transition.

In this section, we analyze the evolution of *routing pathway* during pretraining and introduce two metrics to quantify these changes (Figure 3): (i) the similarity of pathways across different inputs (Section 4.1), and (ii) the consistency of a single input’s pathway (Section 4.2). Importantly, all metrics are computed on pretraining samples that have already been memorized, i.e., those whose training objective has stabilized at low values beyond specific checkpoints, and trace how their internal routing continue to evolve from memorization to generalization. We then assess these metrics as robust generalization monitors (Section 4.3) and conclude by establishing a spectral framework connecting routing geometry to generalization bounds (Section 4.4).

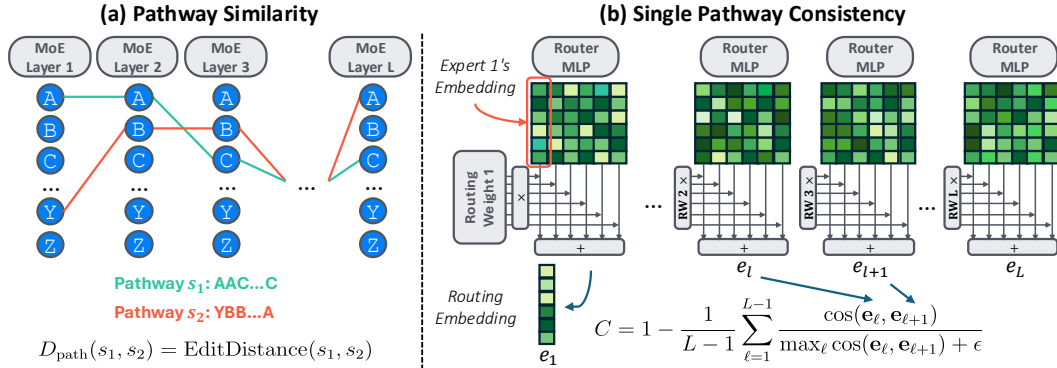


Figure 3: **Pathway Complexity Metrics to Monitor Grokking.** (a) Pathway similarity between samples is measured by edit distance on their sequences of expert choices across layers. (b) Pathway consistency quantifies the smoothness of expert transitions between subsequent layers for the same sample by cosine similarity of their weighted expert embeddings.

4.1 Pathway Distance and Knowledge Sharing between Training Samples

Building on prior findings that grokking coincides with the emergence of *structured internal mechanisms* [15, 22, 31], we hypothesize that as the model begins to generalize, it transitions from highly individualized, input-specific computation toward more *shared, structured computation across related inputs*. This shift should manifest as increasing **pathway similarity**—the convergence of routing patterns across similar samples—indicating that the model is developing common mechanisms for solving related tasks.

We test this by examining how the similarity of expert pathway evolves during pretraining. Each input’s pathway is the ordered sequence of selected experts through the L MoE layers. Specifically, for input x_i , we define its pathway s_i as: $s_i = \text{concat}(e_1^{(i)}, e_2^{(i)}, \dots, e_L^{(i)})$, where $e_\ell^{(i)}$ denotes the ordered list of expert indices at layer ℓ , obtained by ranking experts according to the mean routing